

# BUSINESS PROCESS MANAGEMENT

## 2 – Control Flow Patterns

Christian Salz  
Feb 2025

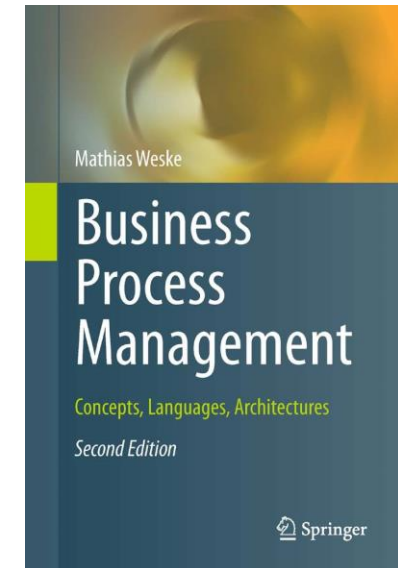
# Starter

- How did Jane manage to make a tennis ball he threw travel a short distance, come to a stop, reverse its direction, and return to his hand without bouncing it off any object or using any strings or attachments?
- Answer: Jane tossed the tennis ball up and down.



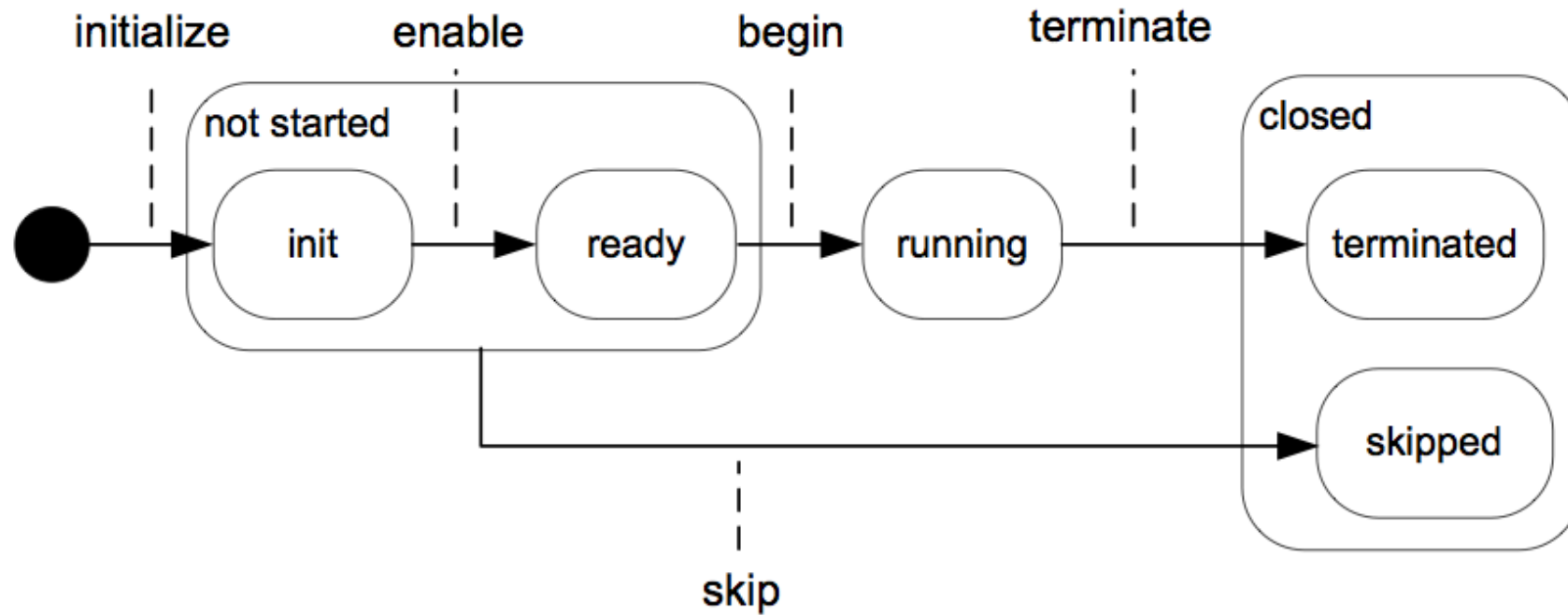
# Evolution of Enterprise System Architectures

- This PowerPoint file contains figures used in
  - Mathias Weske, Business Process Management: Concepts, Languages, Architectures 2012, XIV, 403 p. 300 illus., Hardcover ISBN: 978-3-642-28615-5, eBook ISBN: 978-3-642-28616-2 © Springer-Verlag Berlin Heidelberg 2012, 2007
- This module is based on that book



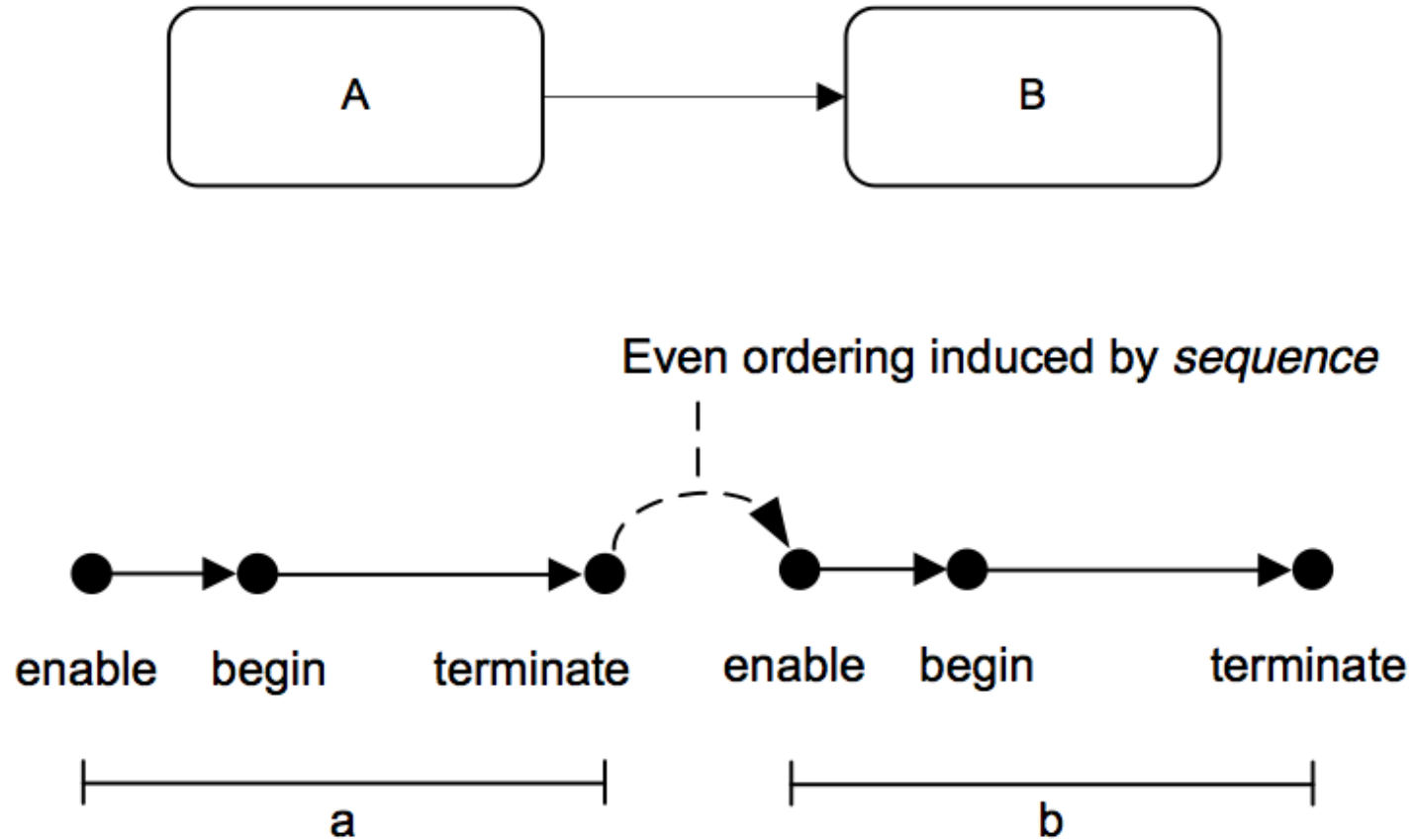
# Contents – Control Flow Patterns

- Sequence
- And Split / And Join; Xor Split / Xor Join; Or Split / Or Join
- Multiple-Merge
- Discriminator
- N-out-of-M Join
- Arbitrary Cycles
- Implicit Termination
- Multiple Instances Without Synchronization
- Multiple Instances With A Priori Design Time / RunTime Knowledge
- Multiple Instances Without A Priori Run Time Knowledge
- Deferred Choice
- Sequential Execution Without A Priori Design Time Knowledge



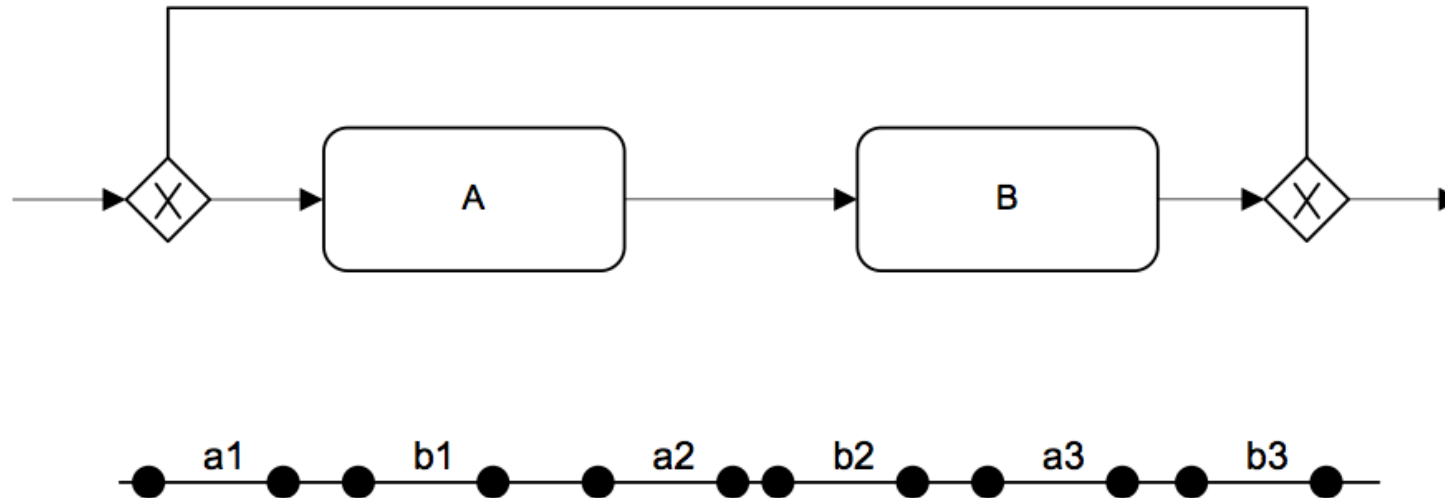
**Fig. 4.1.** State transition diagram for activity instances

## Single instance



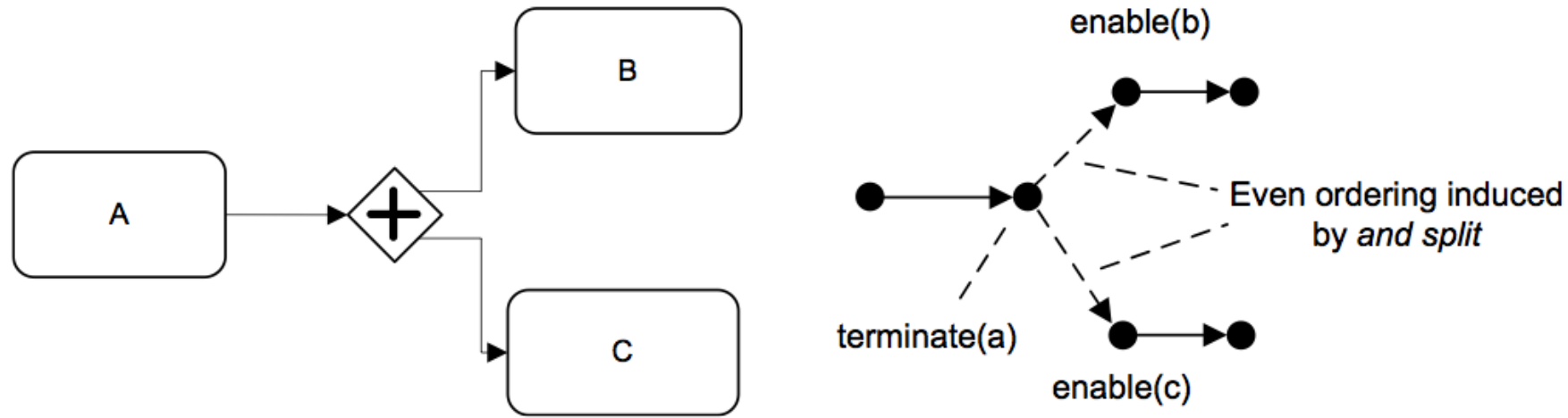
**Fig. 4.2.** Sequence pattern, with event diagram of process instance

# Multiple instances



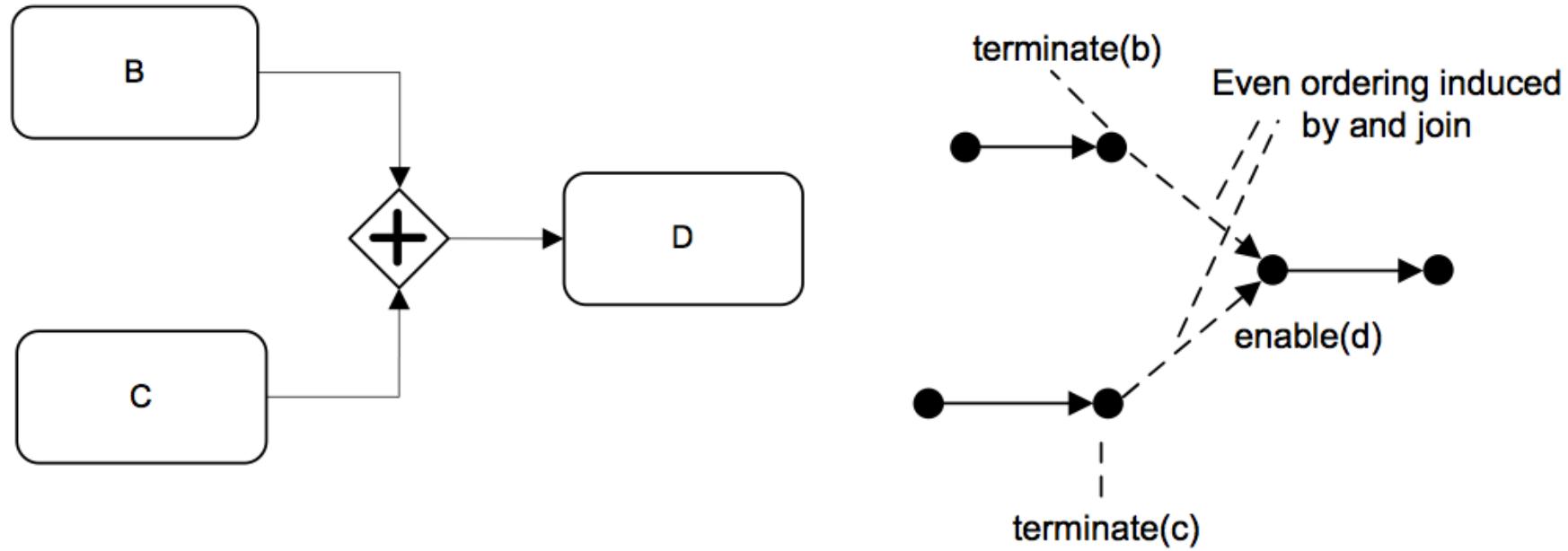
**Fig. 4.3.** Sequence pattern as part of a loop and event diagram showing three loop iterations

# Parallel split



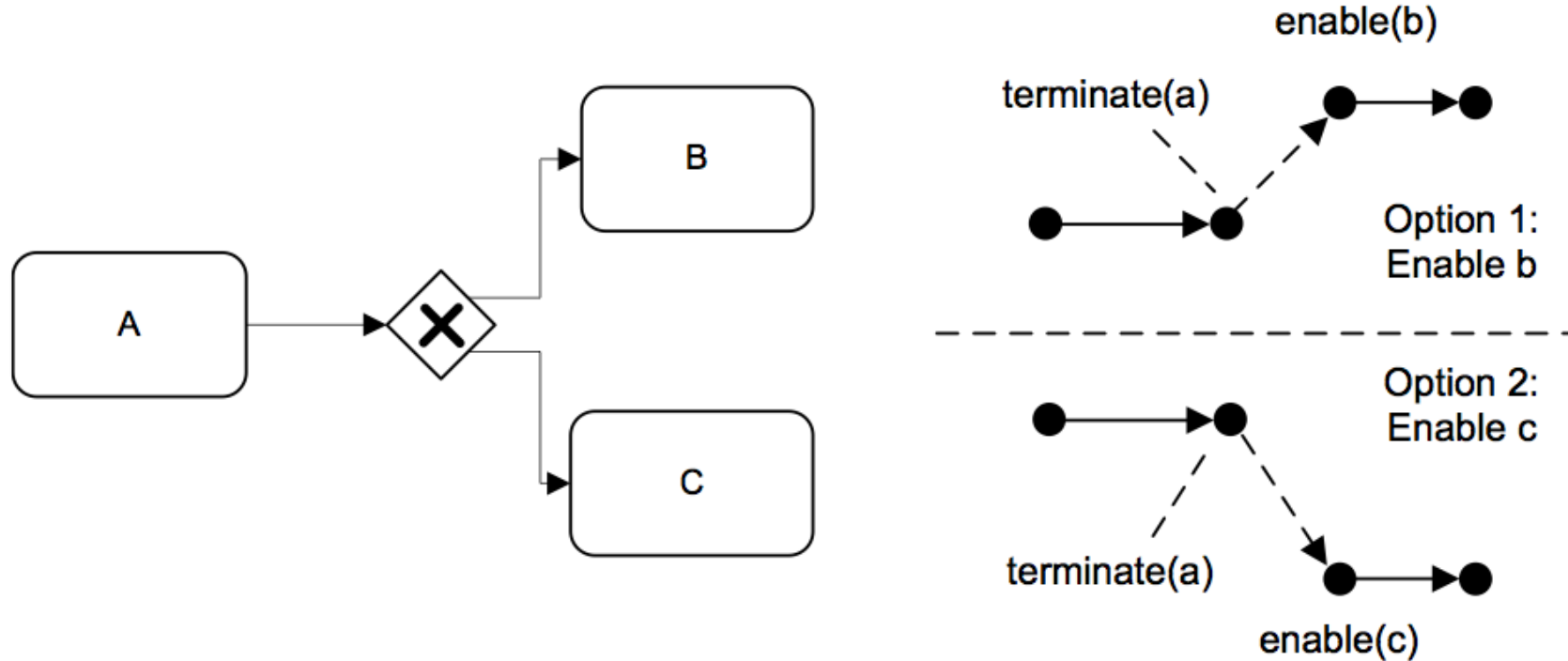
**Fig. 4.4.** And split pattern

# Join



**Fig. 4.5.** And join pattern

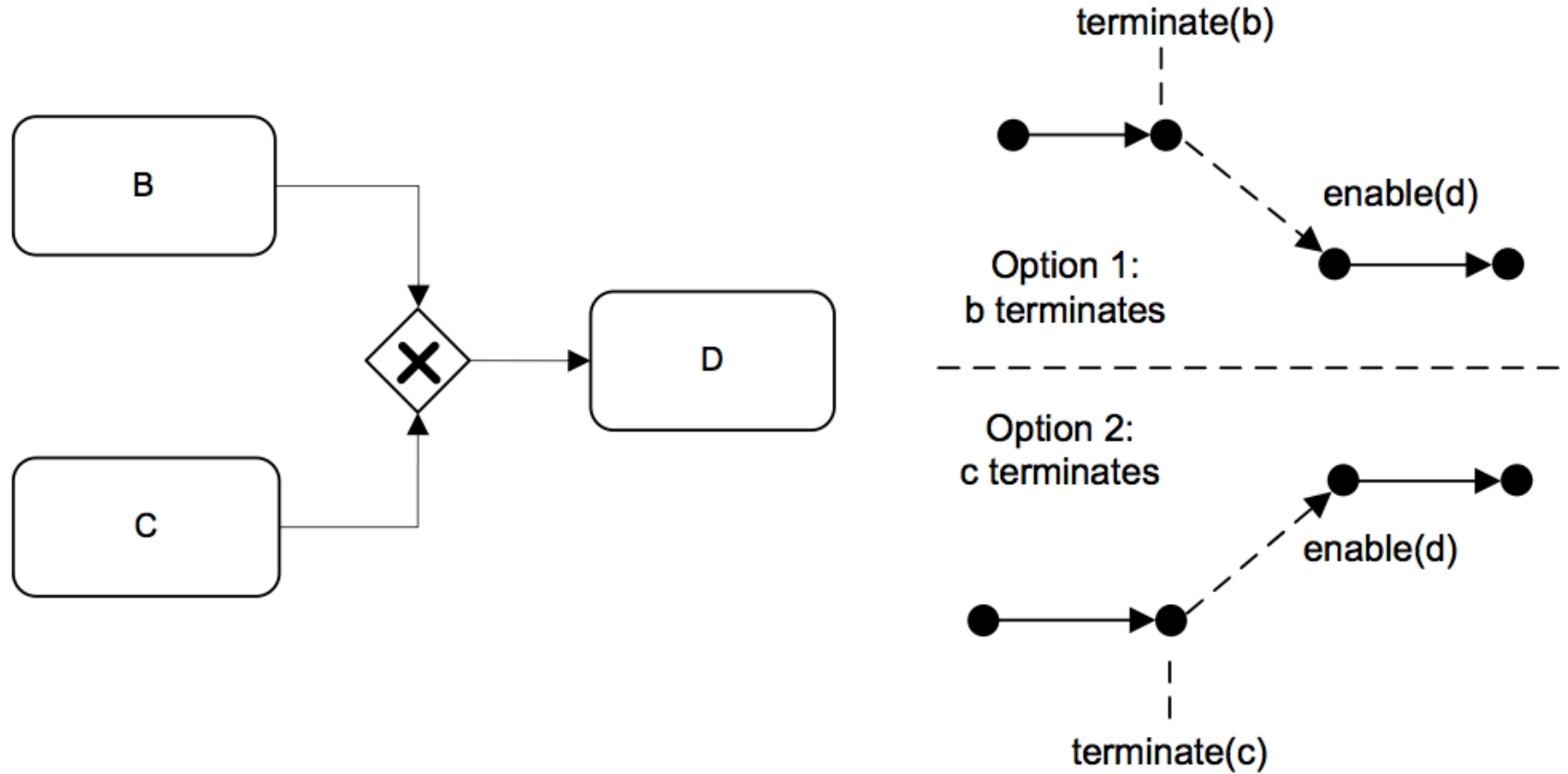
# Cor split or exclusive or split



**Fig. 4.6.** Xor split pattern

- The execution semantics of an exclusive or split determines that for each termination of an activity instance a associated with activity model A there is either an enable event of activity instance b or an enable event of an activity instance c, **but not both**

# Xor join or exclusive or join

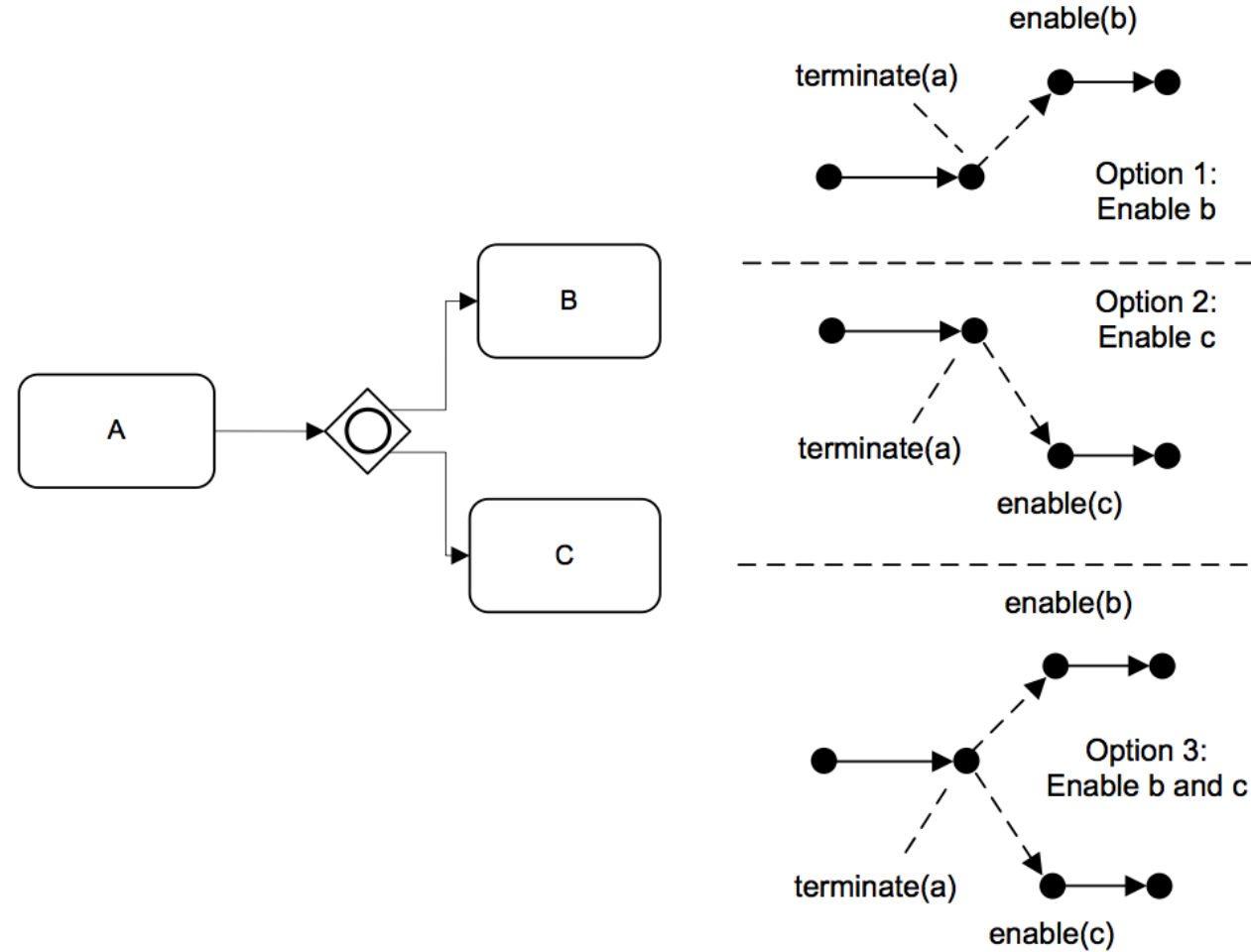


M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

**Fig. 4.7.** Xor join pattern

- For each termination event of an activity instance b or c **there is one and only one enable event** of an activity instance d.

# Or split

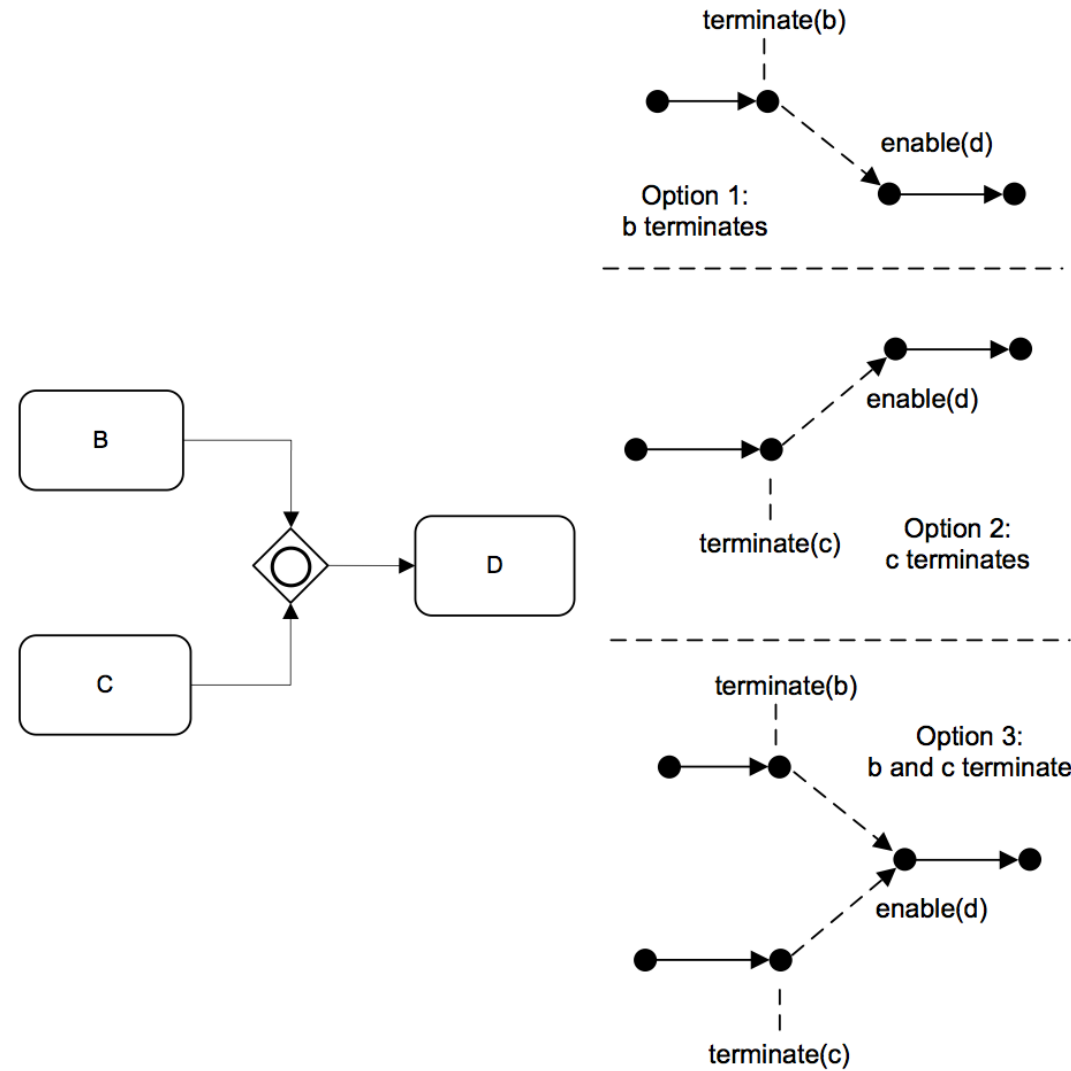


M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

**Fig. 4.8.** Or split pattern

- An or split is a point in a process model where **at least one** branch from a set of branches is chosen.

# Or Join

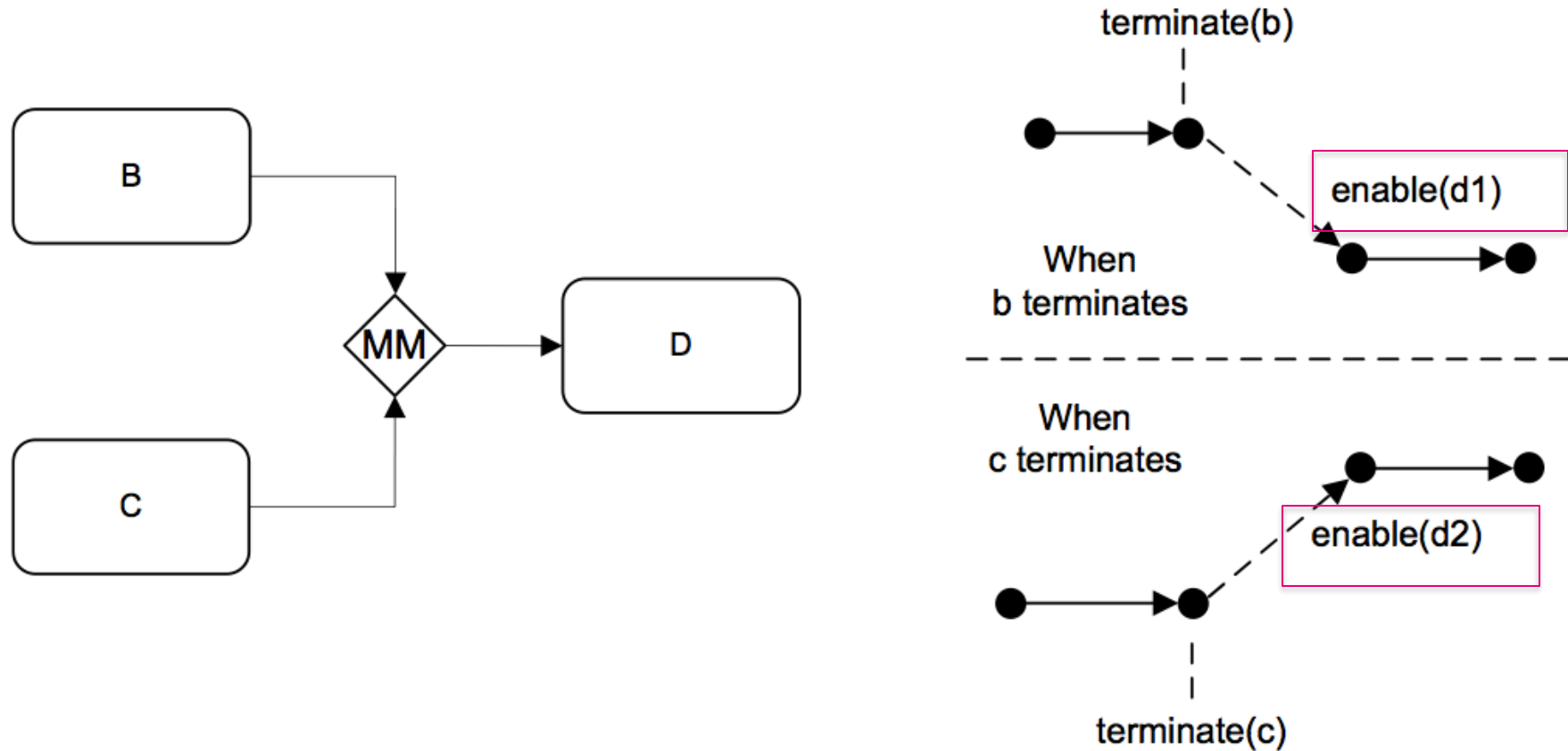


M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

**Fig. 4.9.** Or join pattern

- **Problem:** How long to wait for activation? / **Potential Deadlock** (Waiting Forever) & **Incorrect Execution Order** (Triggering Too Early)

# Multiple merge or multi-merge

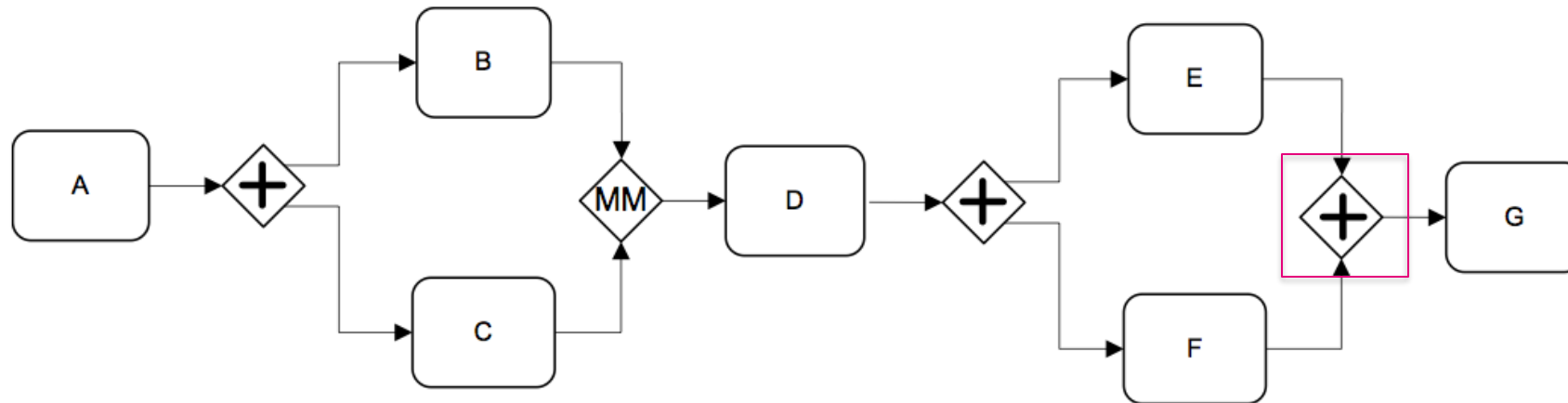


M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

**Fig. 4.10.** Multi-merge pattern

- 2 concurrent threads join without **synchronization**

# Multi-merge synchronization issues (1/2)

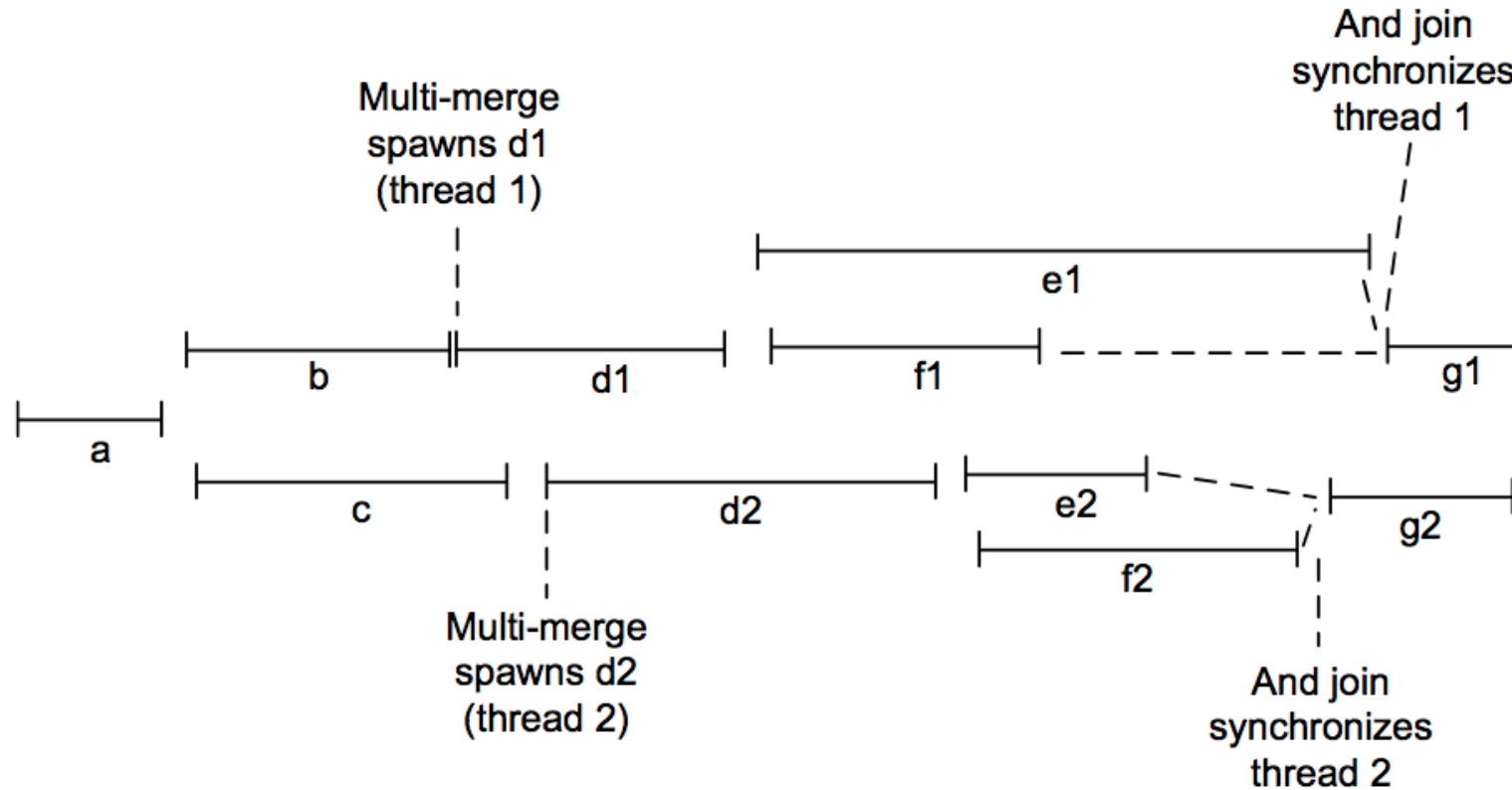


**Fig. 4.11.** Multi-merge example might lead to incorrect synchronization of branches

M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

- Knowing that these belong to different threads of control (f1 belongs to thread 1, while e2 belongs to thread 2), the and join can “decide” **that these threads can not be synchronized**, although they belong to the same process instance.

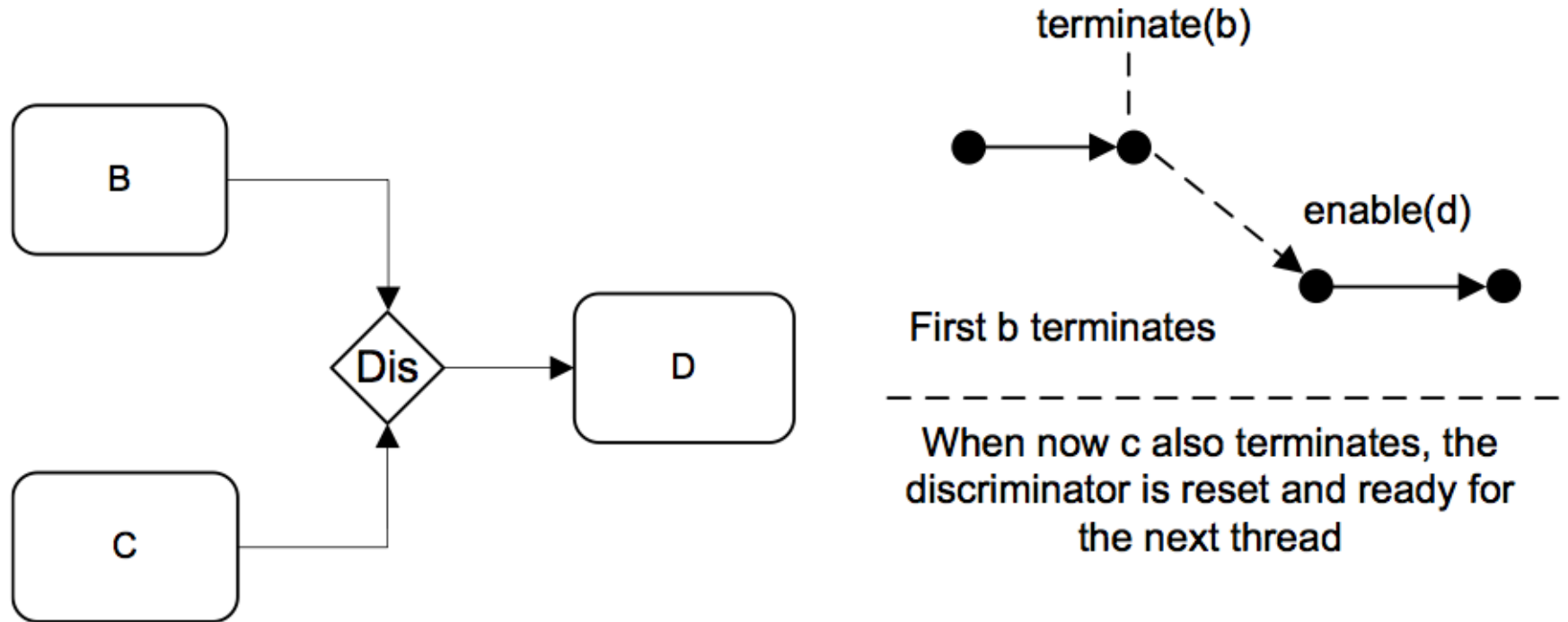
## Multi-merge synchronization issues(2/2)



**Fig. 4.12.** Event diagram of a process instance based on the process model shown in Figure 4.11

- Possible *problem*: termination events on different threads – synchronize f1 with e2?

## Discriminator (1/3)



**Fig. 4.13.** Discriminator pattern

- Wait for one incoming branch to complete. Afterwards wait for all other incoming branches to complete **but ignore them**. Once all incoming branches have been triggered, **it resets itself** so that it can be triggered again

## Discriminator (2/3)

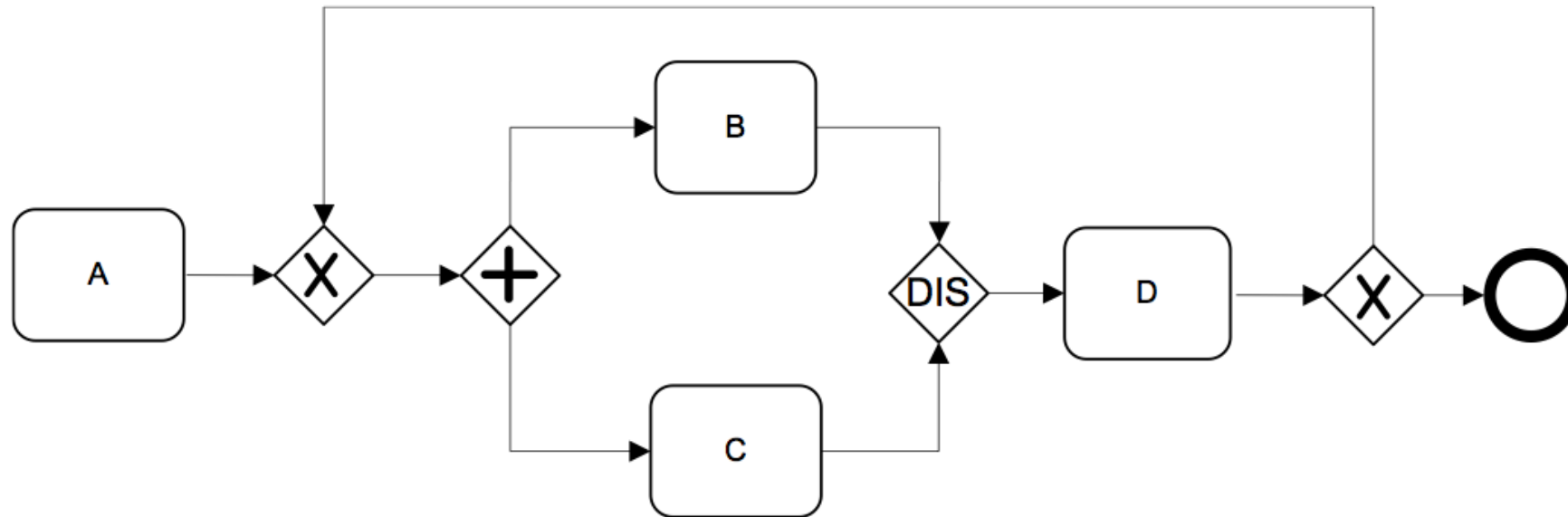
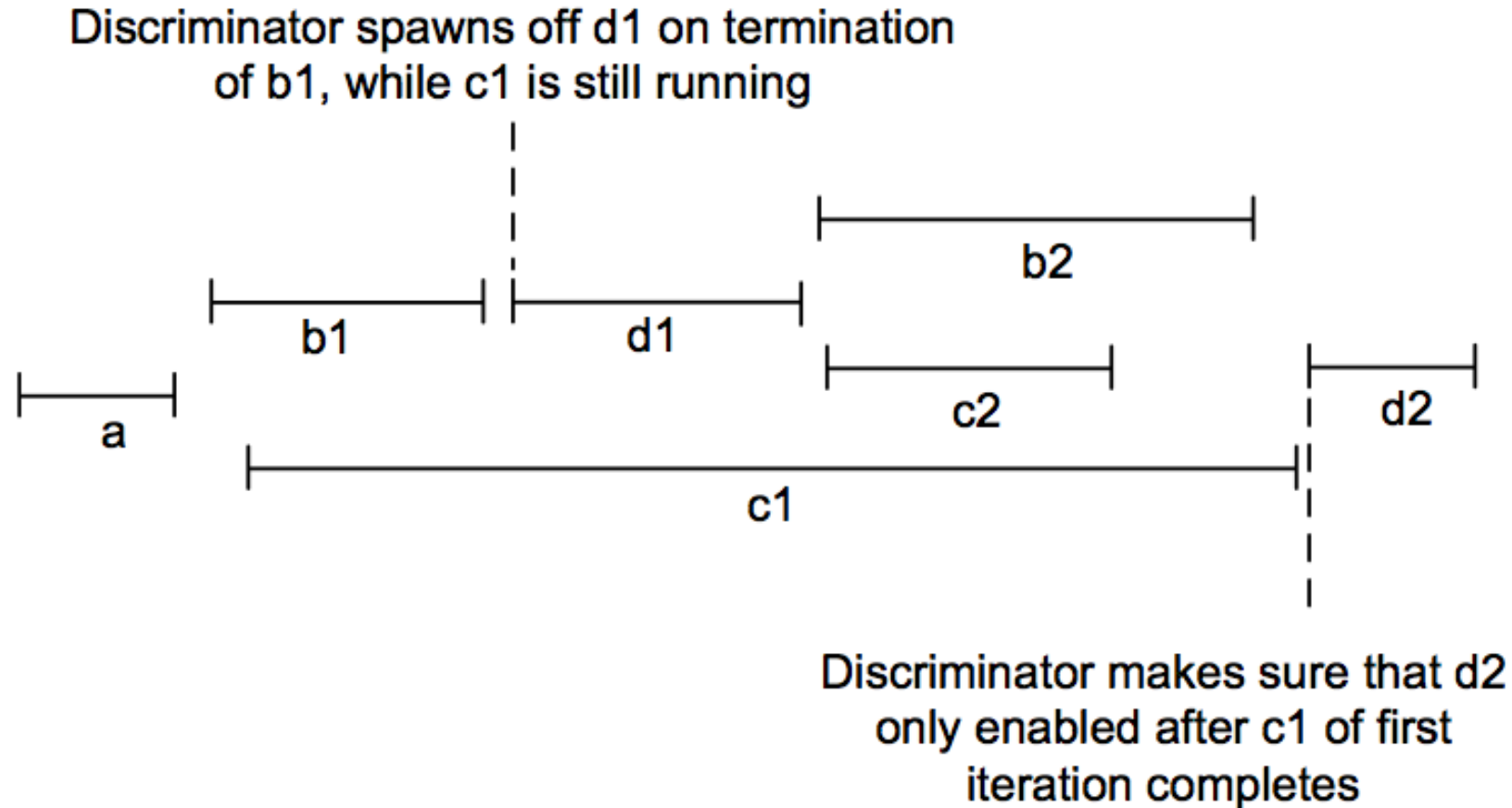


Fig. 4.14. Discriminator example

M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

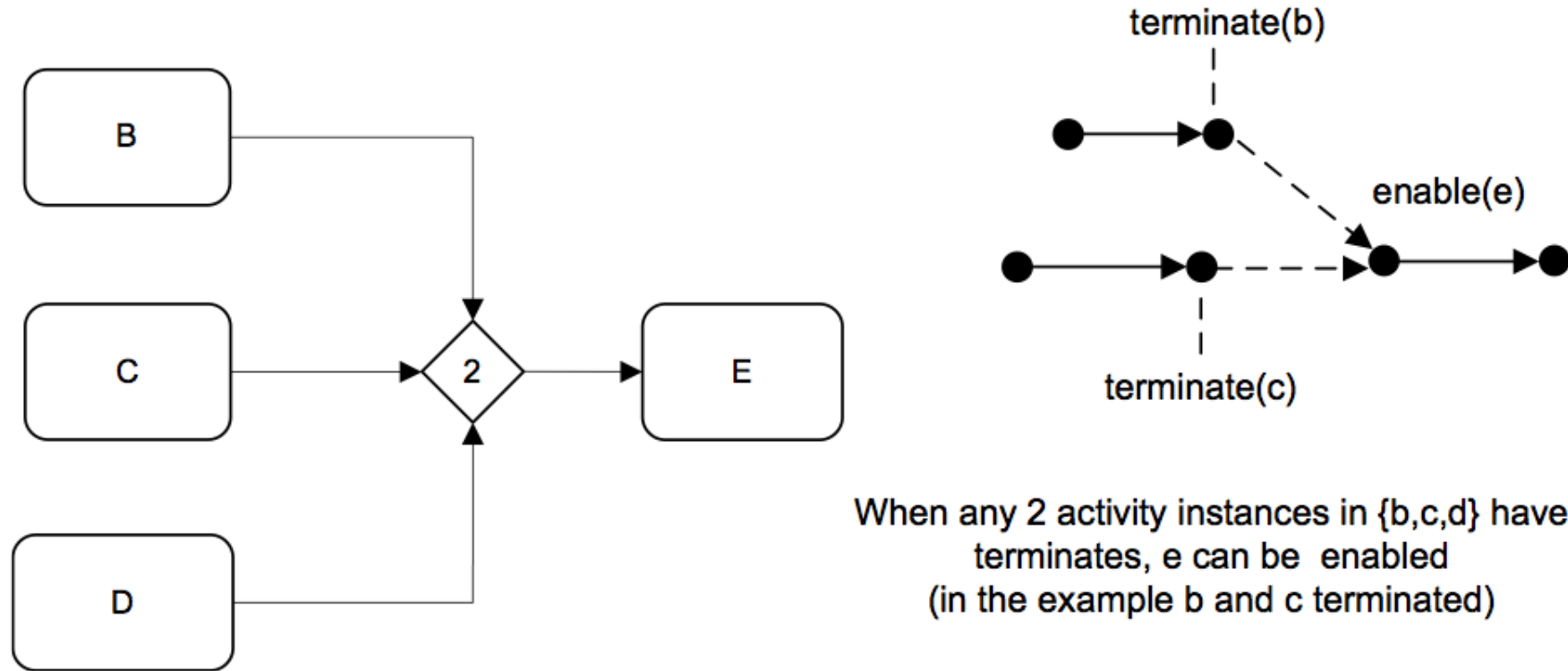
- If gathering the ignored branches were not part of the functional behaviour of the discriminator pattern, **there would be no way to distinguish** a second iteration of a loop from a late branch of its first iteration.

## Discriminator (3/3)



**Fig. 4.15.** Event diagram of discriminator example

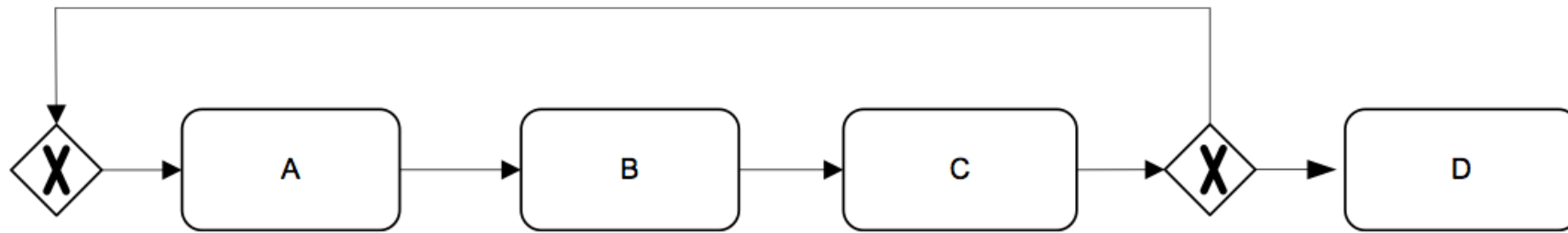
## N-out-of-M join (Discriminator)



**Fig. 4.16.** N-out-of-M join pattern

- The subsequent activity is initiated **after  $N \leq M$  paths have completed** and the respective termination events have occurred.

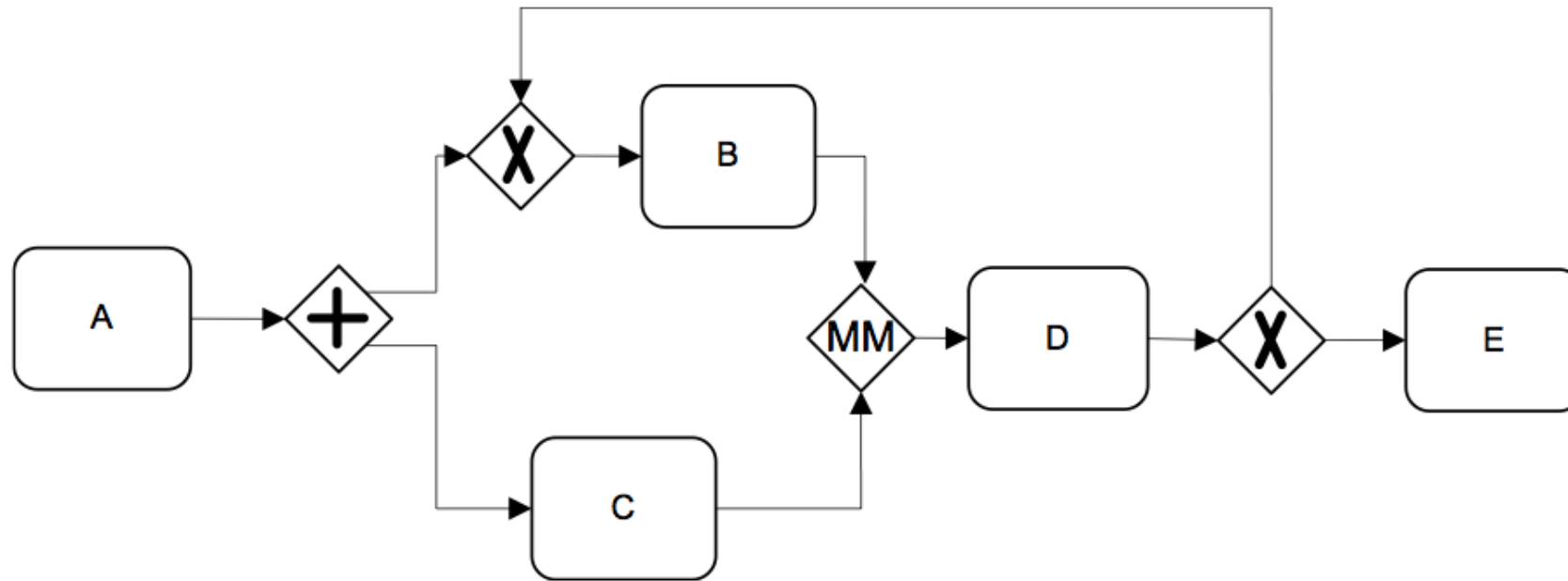
## Arbitrary cycle (1/2)



**Fig. 4.17.** Graphical representation of arbitrary cycles pattern

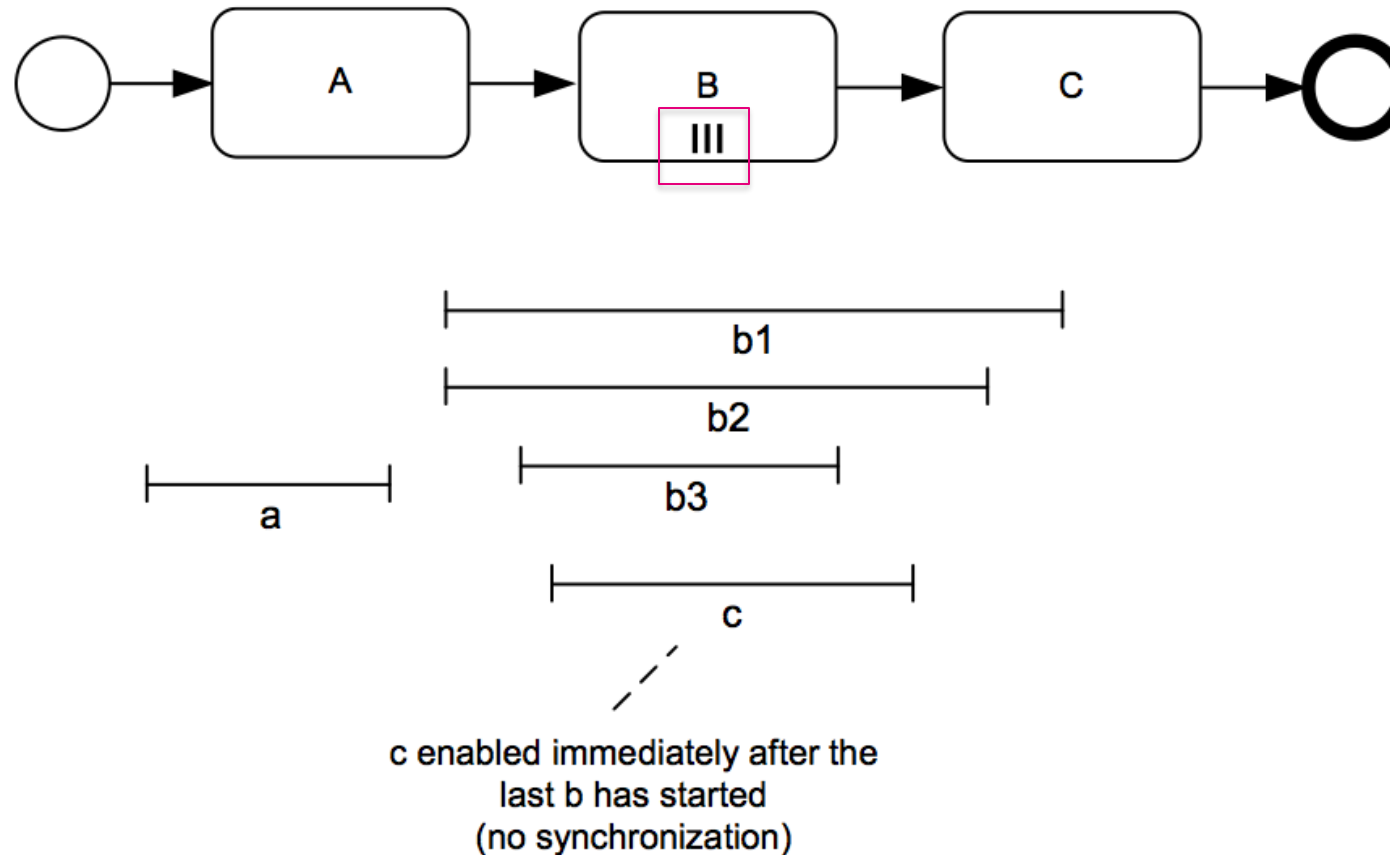
- An arbitrary cycle is a point in a process model where **one or more activities can be executed repeatedly.**

## Arbitrary cycle (2/2)



**Fig. 4.18.** Arbitrary cycles example, using multiple merge pattern

# Multiple Instances Without Synchronization

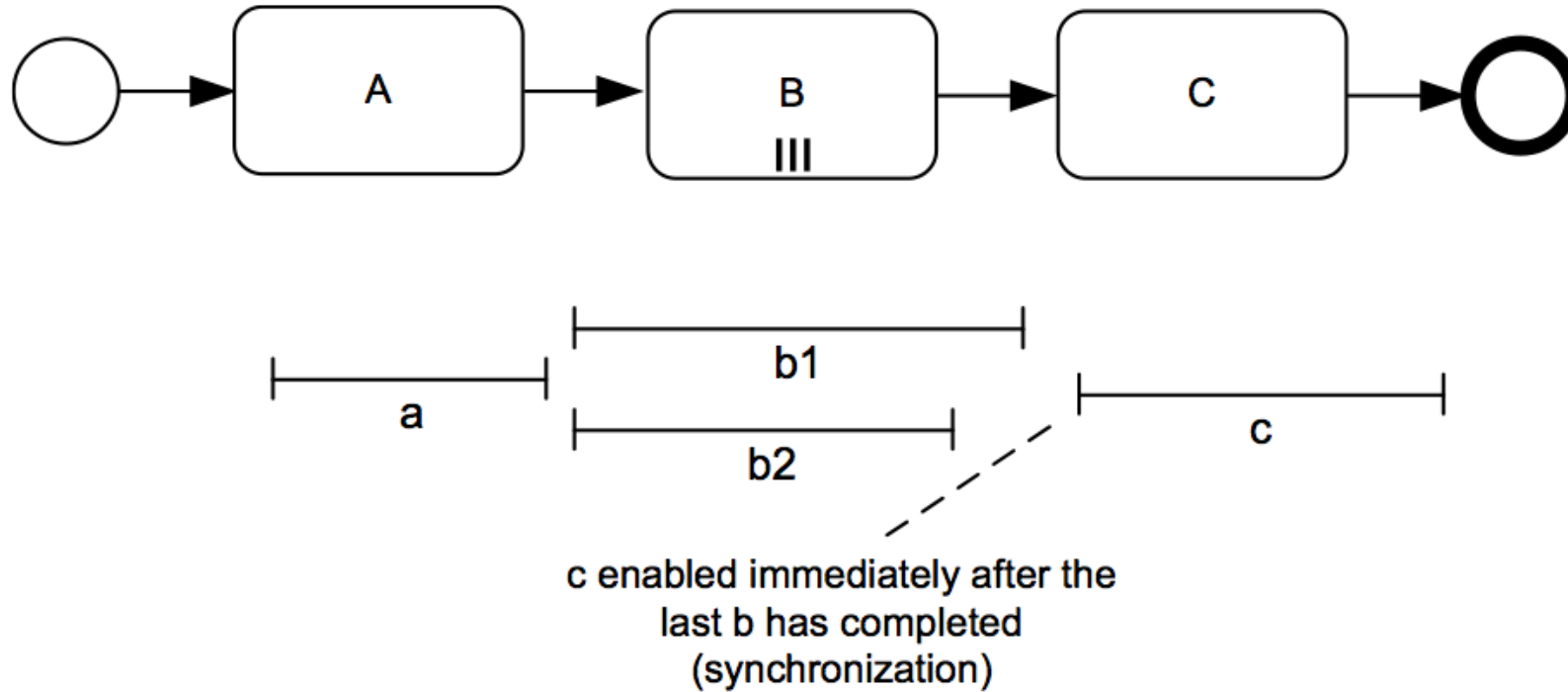


M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

**Fig. 4.19.** Example for multiple instances without synchronization

- The term “**without synchronization**” in the context of this example means that the follow-up activity instance c can be **enabled immediately** after the instances for B have been enabled.

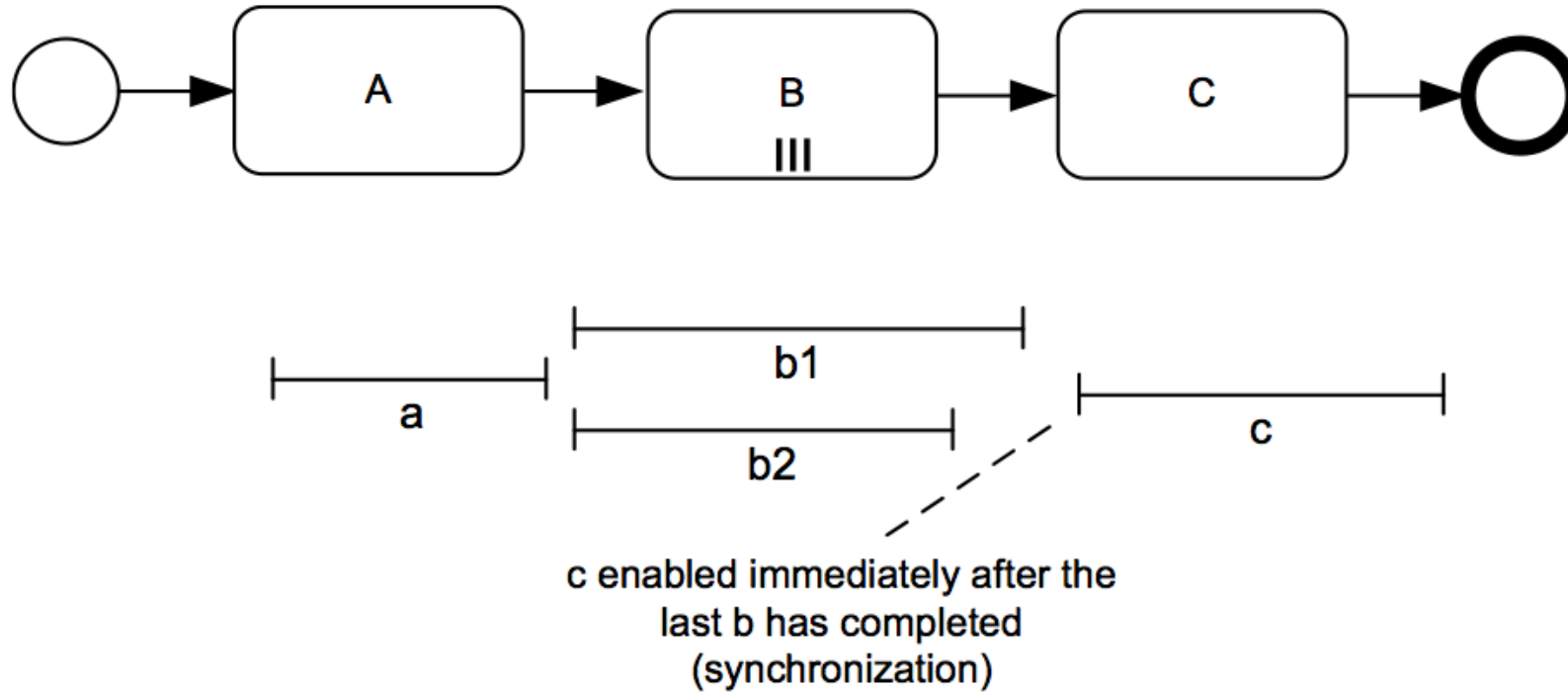
# Multiple instances with a priori design time knowledge



**Fig. 4.20.** Example for multiple instances with a priori design time knowledge

- The number of activity instances of an activity model is known at **design time** (**NrOfInstances** = 2).

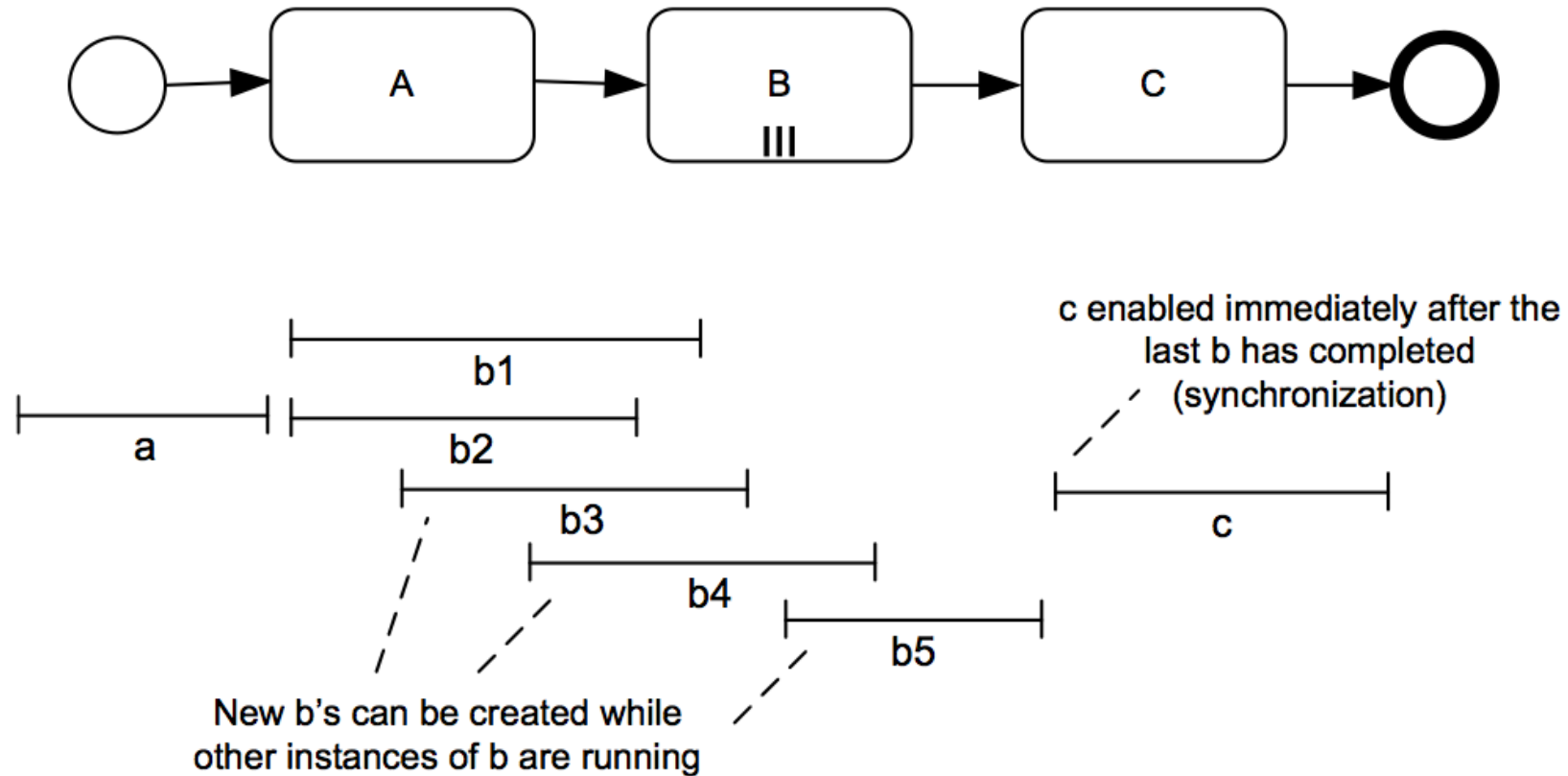
# Multiple Instances With A Priori Run Time Knowledge



**Fig. 4.20.** Example for multiple instances with a priori design time knowledge

- This expression is evaluated during run time to **compute the number of activity instances** for a specific process instance. (NrOfInstances = GetNoOfLineitems(order);)

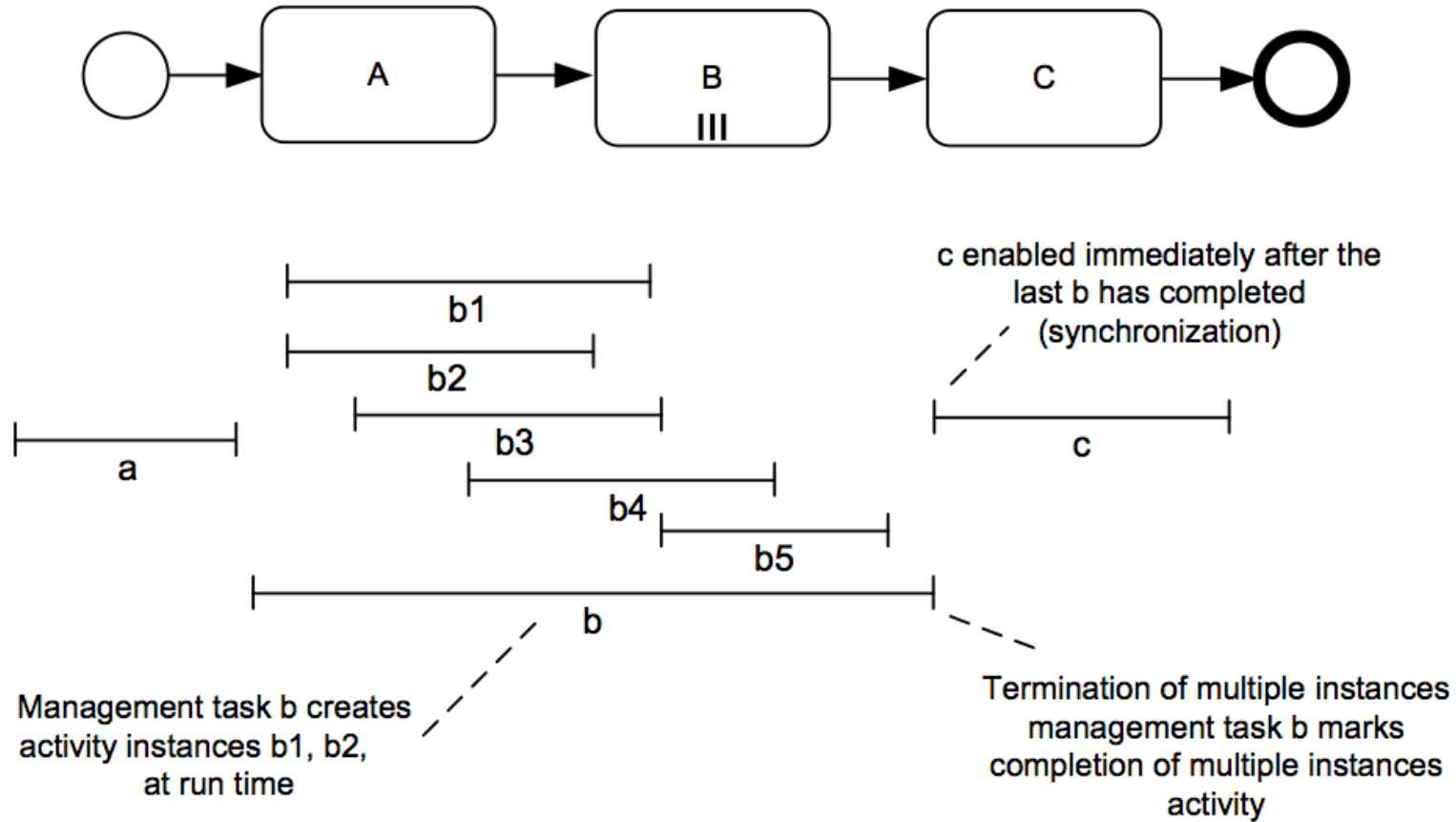
# Multiple Instances Without A Priori Run Time Knowledge



**Fig. 4.21.** Example for multiple instances without a priori run time knowledge pattern

- The number of instances of a given activity is **not known during design time**; nor is it known at any stage during **run time** / Problem: until **what point in time** is it possible to create new activity instances of B?

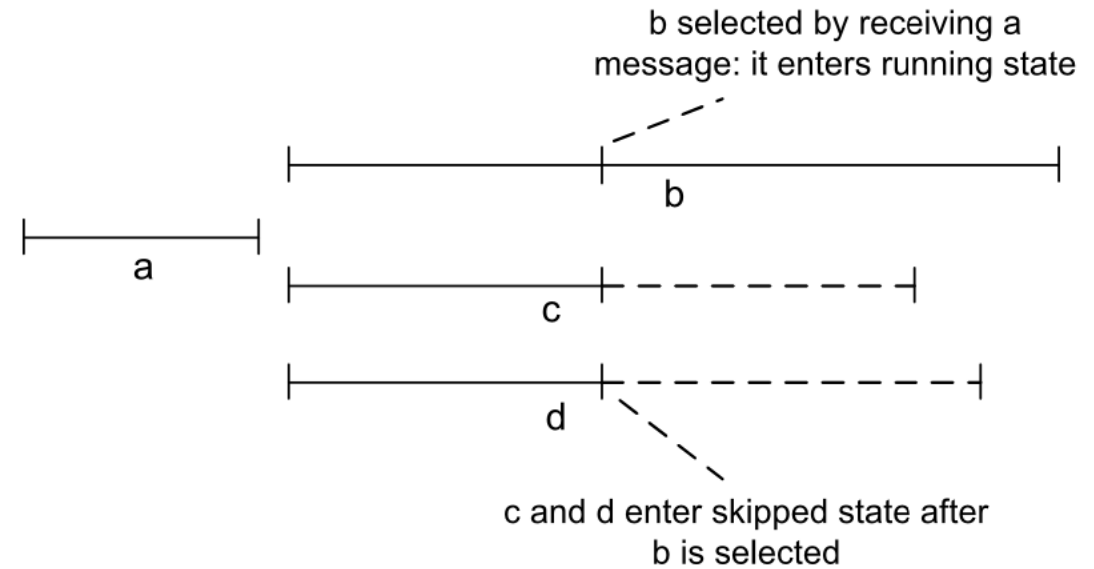
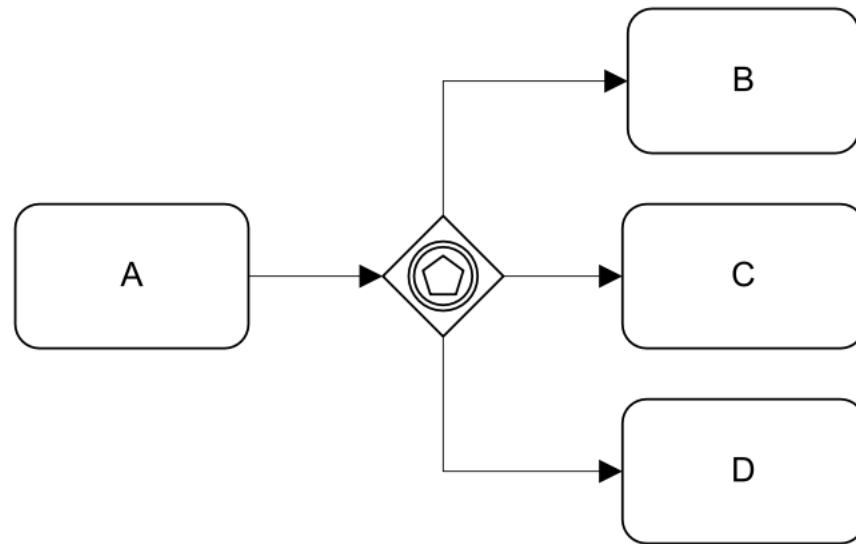
# Management activity related to the multiple instances activity



**Fig. 4.22.** Multiple instances without a priori run time knowledge pattern, including management task

- The management activity explicitly **defines the end** of the multiple instances activity. It is also responsible for **creating new instances** of the multiple instances activity.

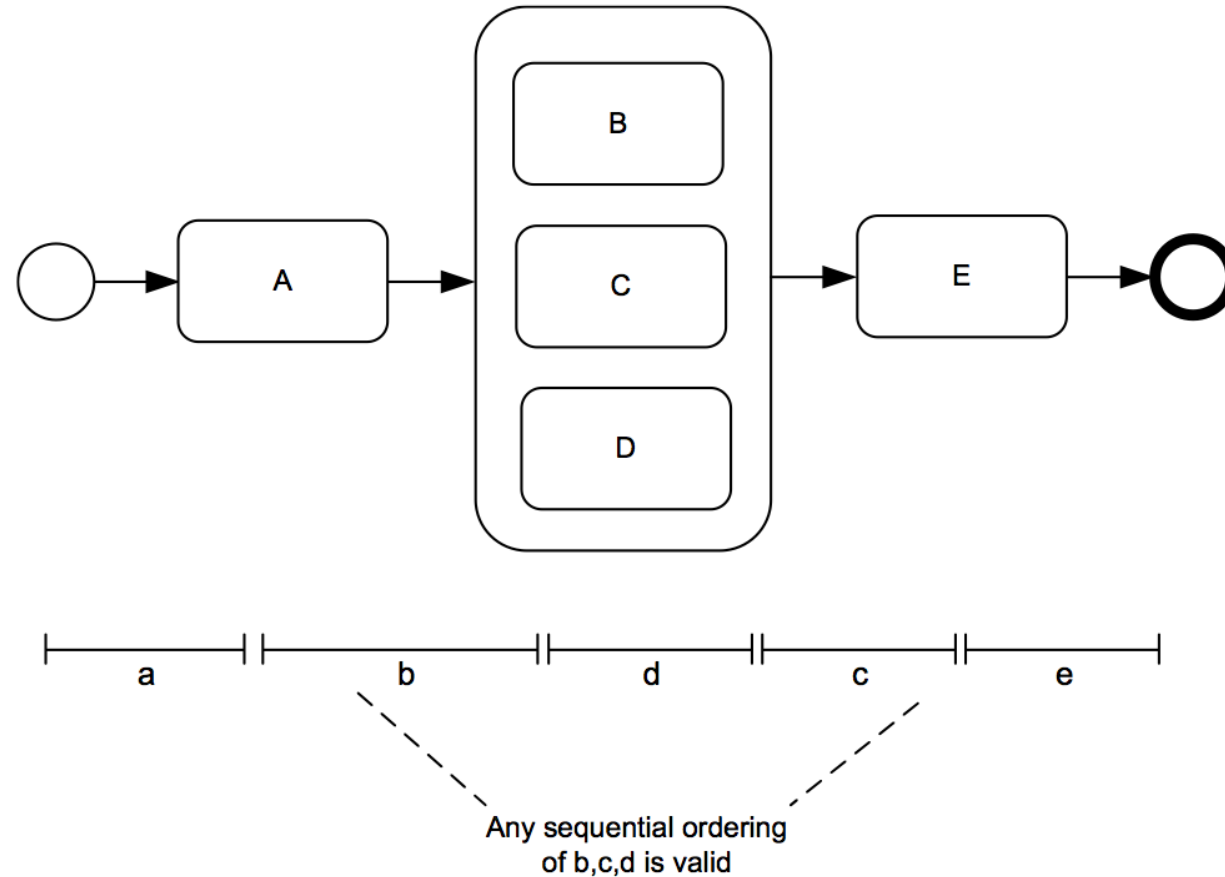
# Deferred choice



**Fig. 4.23.** Example of deferred choice pattern

- State-based patterns capture the implicit behaviour of processes that is based **not on the current case** but rather on the **environment** or other parts of the process (e.G incoming mail)

# Sequential Execution without A Priori Design Time Knowledge



M. Weske: Business Process Management,  
© Springer-Verlag Berlin Heidelberg 2012, 2007

**Fig. 4.24.** Sequential execution without a priori design time knowledge; any sequential execution ordering of *B*, *C*, and *D* is possible

- Activities are executed **sequentially in an order** that is defined while the process instance runs, so any sequential execution of activity instances b, c and d are valid

# Questions?



"[Dieses Foto](#)" von Unbekannter Autor ist lizenziert gemäß [CC BY-NC](#)